MyAnimeList Data Analysis

Alexander Cai

2025-02-28

Introduction:

Idea:

For many years, I have been a large fan of the Japanese medium, anime. The unique style of animation, story telling, and Japanese culture have brought a worldwide audience together. From global hits like Naruto and Attack on Titan, to cult favorites like Code Geass and Fooly Cooly, there is just about an anime for everyone. With every anime series comes different information associated with the show. For example, the airing date, the studio who created the show, the episode length,are just to name a few. As a data scientist, I was curious if I could use the following data to somehow draw conclusions about anime. It was through this idea which lead me to anime's largest yet most popular database, MyAnimeList.

What is MyAnimeList?:

MyAnimeList is a online website anime/manga fans use to mostly track their anime progress, research about new shows, meet others with similar interest, and rate the shows they watched. Personally, I use MyAnimeList for about 5 years, and have mostly used it to log and rate shows, as well as see fellow friends watching history.

Goal:

As a long time user of the platform, I was curious to what sort of data would influence the scoring criteria of anime, as only a handful of anime released every yearly season are given critical acclaim, while many are left with little attention. I will be downloading a CSV file containing all of the information and anime up til 2023 from MyAnimeList, exploring the variables that make up a show, and making statistical summaries with the following data. We will perform the following data analysis on R Studio.

Dataset:

The dataset: I pulled the following dataset off of the popular online platform, Kaggle. The dataset was made by the user SAJID, who compiled the data with a variety of different variables. Link: https://www.kaggle.com/datasets/dbdmobile/myanimelist-dataset

#Rstudio code:

Packeages Used:

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(car)
```

```
## Loading required package: carData
```

```
##
## Attaching package: 'car'
```

```
## The following object is masked from 'package:dplyr':
##
##     recode
```

```
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

Loading Our Dataset:

```
anime.dataset.2023 <- read.csv("C:/Users/aleab/Downloads/Projects/MyAnimeList Data Analysis/anime-dataset-2023.csv", comment.char="#")
summary(anime.dataset.2023)
```

```
##    anime_id             Name           English.name        Other.name
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##     Score              Genres            Synopsis             Type
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Episodes            Aired             Premiered            Status
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Producers          Licensors           Studios             Source
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Duration            Rating              Rank            Popularity
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Favorites          Scored.By           Members           Image.URL
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
```

Part 1) Data Cleaning:

```
#Make a copy for reference:
dataset <-anime.dataset.2023
summary(dataset)
```

```
##    anime_id             Name           English.name        Other.name
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##     Score              Genres            Synopsis             Type
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Episodes            Aired             Premiered            Status
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Producers          Licensors           Studios             Source
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Duration            Rating              Rank            Popularity
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Favorites          Scored.By           Members           Image.URL
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
```

```
dataset$Image.URL <- NULL
dataset$Status <- NULL
dataset$anime_id <- NULL
dataset$Synopsis <- NULL
dataset$Premiered <- NULL
dataset$Other.name <- NULL
dataset$English.name <- NULL
dataset$Licensors <- NULL
dataset$Producers <- NULL
summary(dataset)
```

```
##      Name              Score              Genres              Type
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Episodes            Aired              Studios             Source
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Duration            Rating              Rank            Popularity
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Favorites          Scored.By           Members
##  Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character
```

Deleting Nulls

```
dataset <- na.omit(dataset)
```

Changing variables to numeric instead of class character:

```
dataset[, c("Score", "Episodes", "Rank", "Popularity", "Favorites", "Scored.By", "Members")] <- lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", "Favorites", "Scored.By", "Members")], as.numeric)
```

```
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
## Warning in lapply(dataset[, c("Score", "Episodes", "Rank", "Popularity", : NAs
## introduced by coercion
```

Transform our character classes to Qualative predictors:

```
dataset[, c("Genres", "Type", "Aired", "Studios", "Source", "Duration", "Rating")] <- lapply(dataset[, c("Genres", "Type", "Aired", "Studios", "Source", "Duration", "Rating")], as.factor)
```

Quantifying our "Duration" predictor by unforiming the metric:

First, we shorten our Duration to just show the minutes:

```
dataset$Duration <- gsub(" per ep", "", dataset$Duration)
dataset$Duration <- gsub("min", "", dataset$Duration)
dataset <- na.omit(dataset)
summary(dataset)
```

```
[illegible]
```

```
summary(anime.dataset.2023)
```

```
##    anime_id             Name           English.name        Other.name
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##     Score              Genres            Synopsis             Type
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Episodes            Aired             Premiered            Status
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Producers          Licensors           Studios             Source
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##    Duration            Rating              Rank            Popularity
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##   Favorites          Scored.By           Members           Image.URL
##  Length:20828       Length:20828       Length:20828       Length:20828
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
```

Coverting hours to minutes:

Delete "hr" and turn all the variable numeric.

```
dataset$Duration <- gsub(" hr ", "", dataset$Duration)
dataset$Duration <- as.numeric(dataset$Duration)
```

```
## Warning: NAs introduced by coercion
```

Because our data is currently stored as hour/minute (exp: 1 hour and 42 minutes = 142), let's convert our data to display the accurate run time.

```
dataset$Duration_min <- ifelse(dataset$Duration >= 100, (dataset$Duration %/% 100) * 60 + (dataset$Duration %% 100), dataset$Duration)
[illegible]
```

Checking the new category:

```
dataset <- na.omit(dataset)
summary(dataset$Duration_min)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00   15.00   24.00   29.14   28.00  163.00
```

```
dataset$Duration <- NULL
```

Converting "Airing" to only show year of premire:

```
dataset$Start_Year <- sub(".*, (\\d{4}).*", "\\1", dataset$Aired)
[illegible]
dataset$Start_Year <- as.numeric(dataset$Start_Year)
```

```
## Warning: NAs introduced by coercion
```

```
summary(dataset$Start_Year)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##    1917    1997    2007    2003    2013    2023      75
```

```
dataset$Aired <- NULL
[illegible]
```

Part 2) Exploratory Data Analysis:

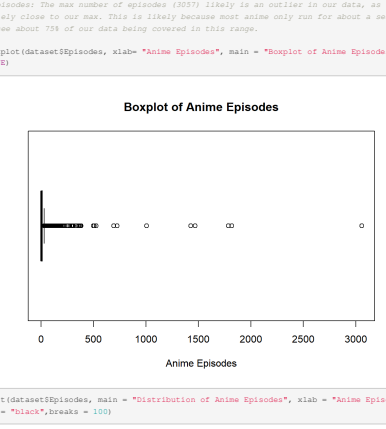
Plotting/insepcting variables:

Score

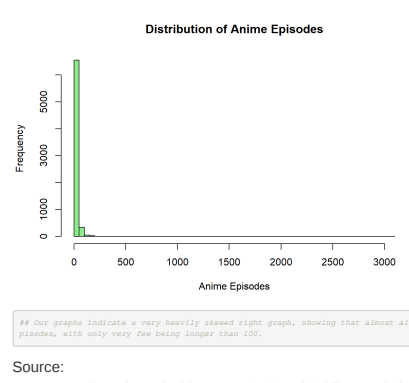
```
summary(dataset$Score)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.850   5.890   6.590   6.566   7.200   9.100
```

```
boxplot(dataset$Score, xlab= "Anime Scores", main = "Boxplot of Anime Scores", col = "blue", horizontal = TRUE)
```



```
hist(dataset$Score, main = "Distribution of Anime Scores", xlab = "Anime Scores", col = "lightblue", border = "black",breaks = 15)
```



Score: The median/mean scores for anime seem to both show that the average of an anime lands around 6.6. There seems to be confounding variables influencing the score of a show. Our Data seems to be somewhat normal, showing signs of a normal distribution, with a slight skew towards the left.

Type

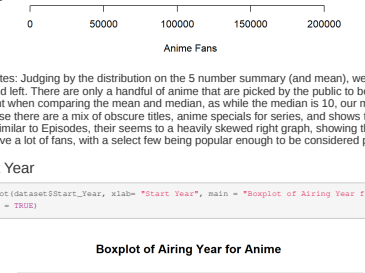
```
summary(dataset)
```

```
[illegible]
```

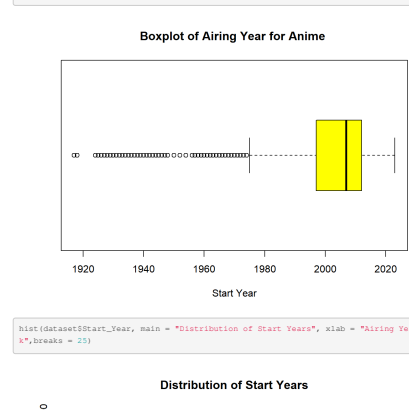
Type: The most popular formats of anime are TV (2744 shows), Movie (1425), and OVA (1245). Both TV and Movie being the most popular formats is understandable. However, seeing the amount of specials (OVA) surprised me. I had never thought about how many were made, especially given almost every popular franchise in anime has about one special per season.

Episodes

```
[illegible]
boxplot(dataset$Episodes, xlab= "Anime Episodes", main = "Boxplot of Anime Episodes", col = "green", horizontal = TRUE)
```



```
hist(dataset$Episodes, main = "Distribution of Anime Episodes", xlab = "Anime Episodes", col = "lightgreen", border = "black",breaks = 100)
```



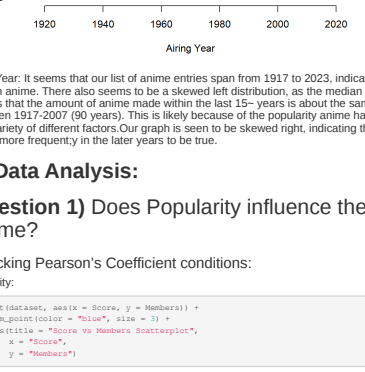
```
[illegible]
```

Source:

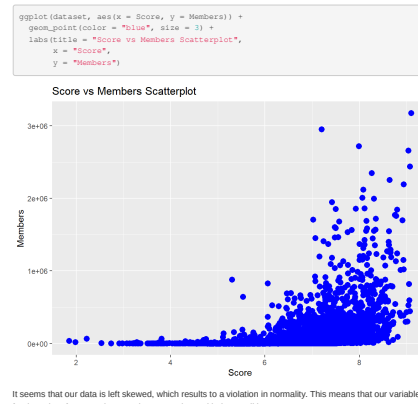
Source: I was pleasently surprised the amount of anime original films. For the longest time, I assumed most anime were covered through Manga and Light Novels, as those are often the shows that get the largest amount of attention. But Looking at the numbers, it almost seems the amount of original source shows are almost equal to those of ones based on manga.

Favorites

```
boxplot(dataset$Favorites, xlab= "Favorited Anime", main = "Boxplot of Number of Fans for an Anime Series", col = "red", horizontal = TRUE)
```



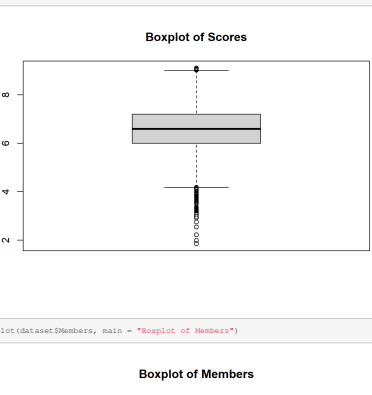
```
hist(dataset$Favorites, main = "Distribution of Fans of an Anime Series", xlab = "Anime Fans", col = "orange", border = "black",breaks = 100)
```



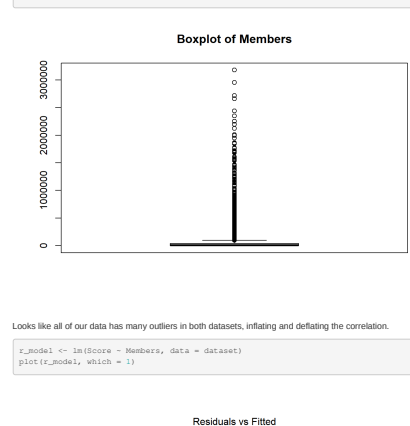
Favorites: Judging by the distribution on the 5 number summary (and mean), we can see that our data is heavily skewed left. There are only a handful of anime that are picked by the public to be their favorites. This is made evident when comparing the mean and median, as while the median is 10, our mean average is 856. This is likely because there are a mix of obscure titles, anime specials for series, and shows that a majority of people do not like. Similar to Episodes, their seems to a heavily skewed right graph, showing that a large majority of anime do not have a lot of fans, with a select few being popular enough to be considered popular amongst people.

Start Year

```
boxplot(dataset$Start_Year, xlab= "Start Year", main = "Boxplot of Airing Year for Anime ", col = "yellow", horizontal = TRUE)
```



```
hist(dataset$Start_Year, main = "Distribution of Start Years", xlab = "Airing Year", col = "pink", border = "black",breaks = 50)
```



Start Year: It seems that our list of anime entries span from 1917 to 2023, indicating a large spread of different eras in anime. There also seems to be a skewed left distribution, as the median is centered around 2007. This means that the amount of anime made within the last 15~ years is about the same as the amount produced between 1917-2007 (90 years). This is likely because of the popularity anime has grown the last few years thanks to a variety of different factors.Our graph is seen to be skewed right, indicating that our suspicion about animes airing more frequently in the later years to be true.

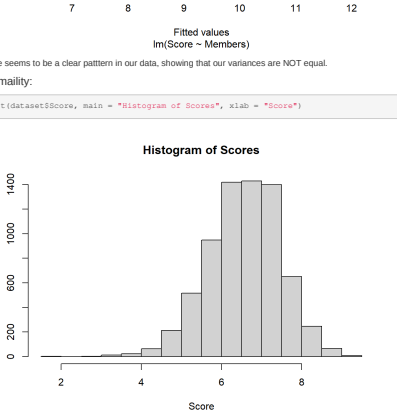
3) Data Analysis:

Question 1) Does Popularity influence the rating of an anime?

Checking Pearson's Coefficient conditions:

Linearity:

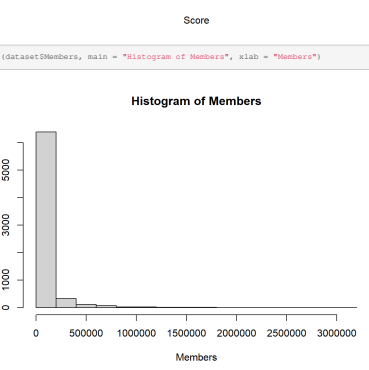
```
ggplot(dataset, aes(x = Score, y = Members)) +
  geom_point(color = "blue", size = 2) +
  labs(title = "Score vs Members Scatterplot",
       x = "Score",
       y = "Members")
```



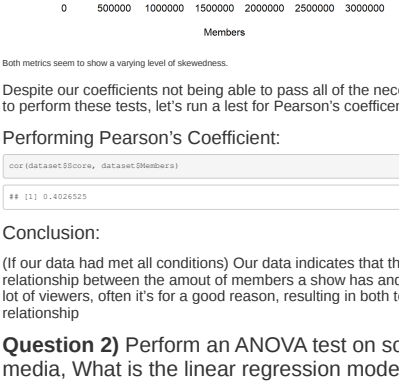
It seems that our data is left skewed, which results to a violation in normality. This means that our variables will likely be misrepresented. However, for the sake of our test, let's continue to push on with the conditions.

Check outliers:

```
boxplot(dataset$Score, main = "Boxplot of Scores")
```

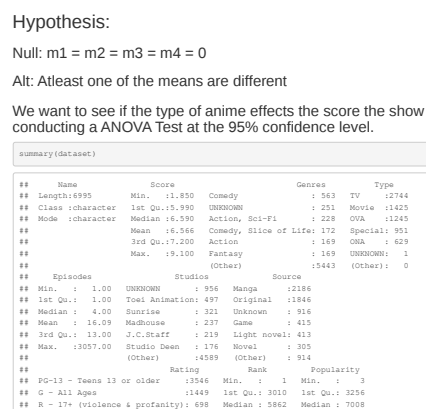


```
boxplot(dataset$Members, main = "Boxplot of Members")
```



Looks like all of our data has many outliers in both datasets, inflating and deflating the correlation.

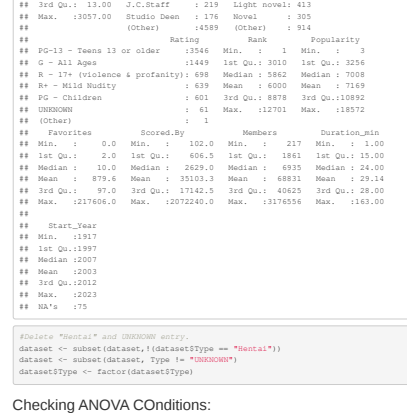
```
s_model <- lm(Score ~ Members, data = dataset)
plot(s_model, which = 1)
```



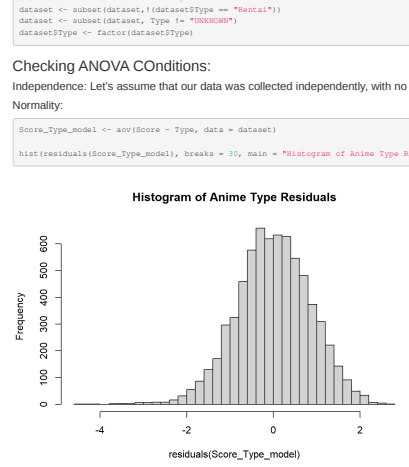
There seems to be a clear pattern in our data, showing that our variances are NOT equal.

Normality:

```
hist(dataset$Score, main = "Histogram of Scores", xlab = "Score")
```



```
hist(dataset$Members, main = "Histogram of Members", xlab = "Members")
```



Both metrics seem to show a varying level of skewedness.

Despite our coefficients not being able to pass all of the necessary conditions needed to perform these tests, let's run a test for Pearson's coefficent as a "what-if"

Performing Pearson's Coefficient:

```
cor(dataset$Score, dataset$Members)
```

```
## [1] 0.4026033
```

Conclusion:

(If our data had met all conditions) Our data indicates that there is a postitive relationship between the amout of members a show has and it's score. When a has a lot of viewers, often it's for a good reason, resulting in both to have a semi-strong relationship

Question 2) Perform an ANOVA test on scores of different media, What is the linear regression model like?

Hypothesis:

Null: m1 = m2 = m3 = m4 = 0

Alt: Atleast one of the means are different

We want to see if the type of anime effects the score the show recieves. We will be conducting a ANOVA Test at the 95% confidence level.

```
summary(dataset)
```

```
[illegible]
```

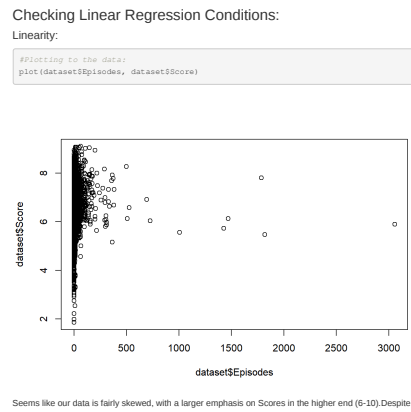
```
#Delete "Music" and UNKNOWN entry:
dataset <- subset(dataset,!(dataset$Type == "Music"))
dataset <- subset(dataset, Type != "UNKNOWN")
dataset$Type <- factor(dataset$Type)
```

Checking ANOVA COnditions:

Independence: Let's assume that our data was collected independently, with no repeated entries.

Normality:

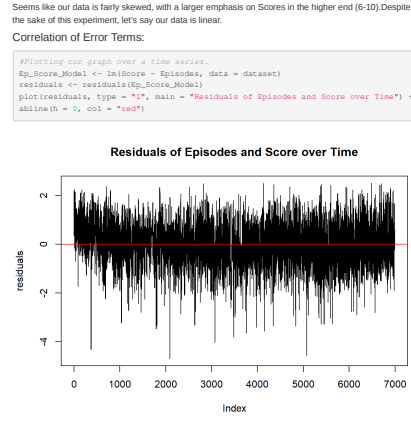
```
Score_Type_model <- aov(Score ~ Type, data = dataset)

hist(residuals(Score_Type_model), breaks = 30, main = "Histogram of Anime Type Residuals")
```



It seems like our data is fairly normally distributed here, with our histogram representing roughly a bell-curve.

Constant Variance(Homoscedasticity):

```
plot(fitted(Score_Type_model), resid(Score_Type_model),
     xlab = "Predicted Values", ylab = "Residuals",
     main = "Residuals vs Predicted Values for Anime Type")
```



There does not seem to a funnel-like pattern for our residuals, and the scatter for each factor(different type) is randomly scattered across our axis.

We can successfully perform ANOVA given the following conditions were passed, we can compute ANOVA:

```
model <- lm(Score ~Type, data = dataset)
anova(model)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
	<int>	<dbl>	<dbl>	<dbl>	<dbl>
Type	4	491.9094	122.9773570	165.8662	7.03667e-136
Residuals	6989	5181.8182	0.7414246	NA	NA

2 rows

```
[illegible]
```

Examining the Linear Model:

```
lm(Score ~Type, data = dataset)
```

```
##
## Call:
## lm(formula = Score ~ Type, data = dataset)
##
## Coefficients:
## (Intercept)       TypeONA       TypeOVA   TypeSpecial        TypeTV
##      6.40816      -0.18631      -0.12772       0.07876       0.44655
```

Conclusions:

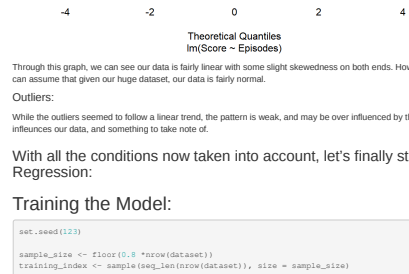
see that movie has been used as the intercept, because R chooses the first category alphabetically. OUr model shows when a show is either a ONA or OVA, it is predicted to score less than in comparison to movies, while Specials and TV Shows often score higher.

Question 3) Create a predictive linear regression model to see if number of episodes can predict a rating of an anime. Conduct a test at a 95% confidence level, and make a prediction of the score of a show given 12 episodes of anime.

Checking Linear Regression Conditions:

Linearity:

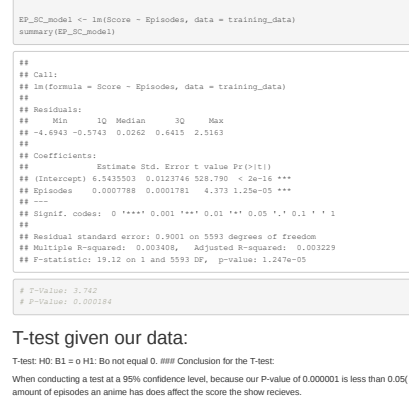
```
#Plotting all the data:
plot(dataset$Episodes, dataset$Score)
```



Seems like our data is fairly skewed, with a larger emphasis on Scores in the higher end (6-10) Despite this, there are small signs of linearity. For the sake of this experiment, let's say our data is linear.

Correlation of Error Terms:

```
#Plotting our graph over a time series:
Ep_Score_Model <- lm(Score ~ Episodes, data = dataset)
residuals <- residuals(Ep_Score_Model)
plot(residuals, type = "l", main = "Residuals of Episodes and Score over Time") +
abline(h = 0, col = "red")
```

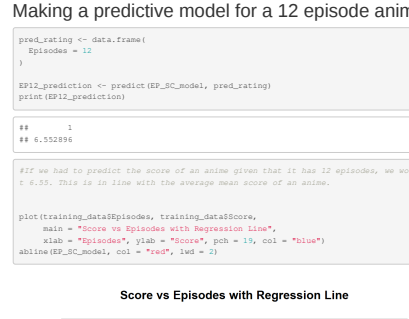


```
## integer(0)
```

Seems there is no clear patter, therefore we can assume that our data's residuals are independent.

Constant errors(Homoscedasticity):

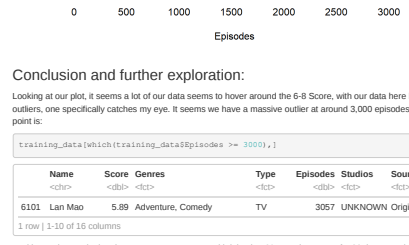
```
plot(Ep_Score_Model, which = 1)
```



Given that our data while roughly may represent a fan-like pattern, is not extreme enough to fail the constant errors condition.

Normality of Residuals:

```
plot(Ep_Score_Model, which = 2)
```



Through this graph, we can see our data is fairly linear with some slight skewedness on both ends. However, through Central Limit Theorem, we can assume that given our huge dataset, our data is fairly normal.

Outliers:

While the outliers seemed to follow a linear trend, the pattern is weak, and may be over influenced by these outliers. Outliers can be a issue that influences our data, and something to take note of.

With all the conditions now taken into account, let's finally start applying linear Regression:

Training the Model:

```
set.seed(123)

sample_size <- floor(0.7 *nrow(dataset))
training_index <- sample(seq_len(nrow(dataset)), size = sample_size)
training_data <- dataset[training_index, ]
test_data <- dataset[-training_index, ]

EP_SC_model <- lm(Score ~ Episodes, data = training_data)
summary(EP_SC_model)
```

```
##
## Call:
## lm(formula = Score ~ Episodes, data = training_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.6943 -0.5743  0.0262  0.6413  2.5163
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 6.5420003  0.0123746 528.790  < 2e-16 ***
## Episodes    0.0007789  0.0001781   4.373 1.25e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9001 on 5593 degrees of freedom
## Multiple R-squared:  0.003408,	Adjusted R-squared:  0.003229
## F-statistic: 19.12 on 1 and 5593 DF,  p-value: 1.247e-05
```

```
[illegible]
```

T-test given our data:

T-test: H0: B1 = 0 H1: B1 not equal 0. ### Conclusion for the T-test:

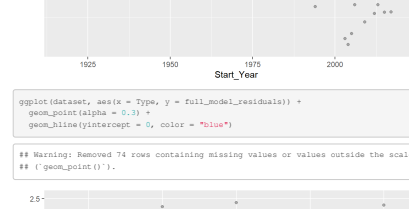
When conducting a test at a 95% confidence level, because our P-value of 0.000001 is less than 0.05(1- .95), we reject the Null Hypothesis. The amount of episodes an anime has does affect the score the show recieves.

Making a predictive model for a 12 episode anime:

```
pred_rating <- data.frame(
  Episodes = 12
)

EP12_prediction <- predict(EP_SC_model, pred_rating)
print(EP12_prediction)
```

```
##        1
## 6.551293
```

```
[illegible]

plot(training_data$Episodes, training_data$Score,
     main = "Score vs Episodes with Regression Line",
     xlab = "Episodes", ylab = "Score", pch = 19, col = "blue")
abline(EP_SC_model, col = "red", lwd = 2)
```



Conclusion and further exploration:

Looking at our plot, it seems a lot of our data seems to hover around the 6-8 Score, with our data here being skewed right. When looking at outliers, one specifically catches my eye. It seems we have a massive outlier at around 3,000 episodes. Out of curiosity, let's discover what this point is:

```
training_data[which(training_data$Episodes >= 3000),]
```

	Name	Score	Genres	Type	Episodes	Studios	Source	Rating	Rank
	<chr>	<dbl>	<fct>	<fct>	<dbl>	<fct>	<fct>	<fct>	<dbl>
6101	Lan Mao	5.89	Adventure, Comedy	TV	3057	UNKNOWN	Original	PG - Children	9305

1 row | 1-10 of 15 columns

Looking at the result, the show "Lan Mao" appears, with it having 3057 and a score of 5.89. It seems that this is our outlier in our training data.

Question 4) Fit a multiple linear regression model, with variables: Y= Score, B1 = Start_Year, B2 = Type, B3 = members.

Making all models:

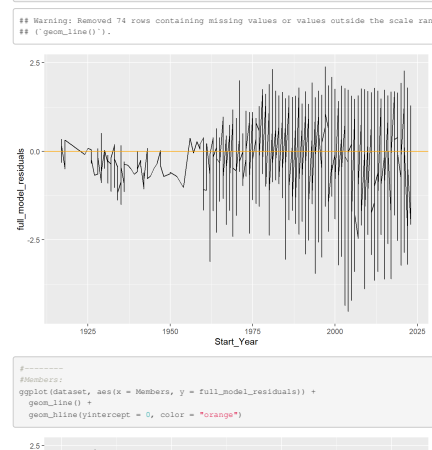
```
Y = dataset$Score
x1 = dataset$Start_Year
x2 = dataset$Type
x3 = dataset$Members
#----------------------
#Building all the ML
x1_x2_x3_model <- lm(Y ~ x1 + x2 +x3, data = dataset, na.action = na.exclude)
#-----------
x1_x2_model <- lm(Y~ x1 +x2, data = dataset)
x1_x3_model <- lm(Y~ x1 +x3, data = dataset)
x2_x3_model <- lm(Y~ x2 +x3, data = dataset)
#----------
x1model <- lm(Y~ x1, data = dataset)
x2model <- lm(Y~ x2, data = dataset)
x3model <- lm(Y~ x3, data = dataset)
```

Checking Conditions for Linear Regression:

Checking Linearity:

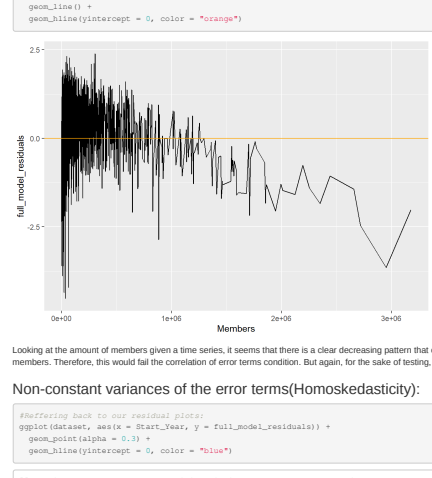
```
dataset$full_model_residuals <- resid(x1_x2_x3_model)

ggplot(dataset, aes(x = Start_Year, y = full_model_residuals)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "blue")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



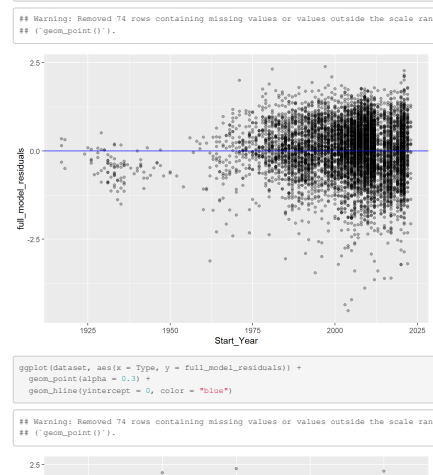
```
ggplot(dataset, aes(x = Type, y = full_model_residuals)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "blue")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



```
ggplot(dataset, aes(x = Members, y = full_model_residuals)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "blue")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

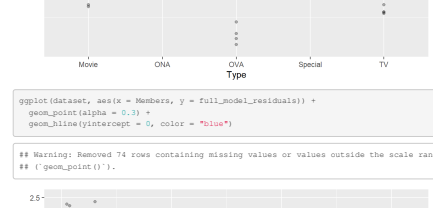


We see a fan-like pattern for our start_year predictor. Therefore, we should drop the predictor. We will keep it in however, for the sake of testing.

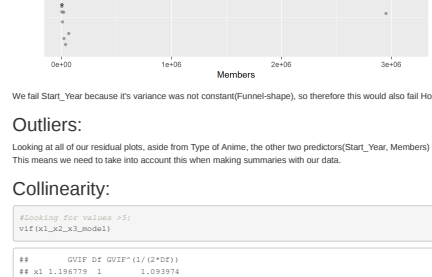
Correlation of Error Terms:

```
#Plotting it over a time series(using the ID# variable):
dataset$index <- 1:nrow(dataset)
#-------
#Start Year:
ggplot(dataset, aes(x = Start_Year, y = full_model_residuals)) +
  geom_line() +
  geom_hline(yintercept = 0, color = "orange")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_line()`).
```



```
#------
#Members:
ggplot(dataset, aes(x = Members, y = full_model_residuals)) +
  geom_line() +
  geom_hline(yintercept = 0, color = "orange")
```

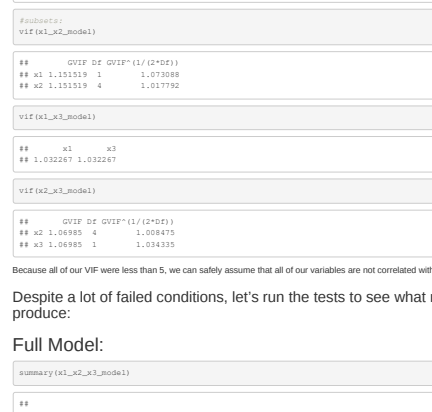


Looking at the amount of members given a time series, it seems that there is a clear decreasing pattern that occurs as we increase thew amount of members. Therefore, this would fail the correlation of error terms condition. But again, for the sake of testing, let's keep it in.

Non-constant variances of the error terms(Homoskedasticity):

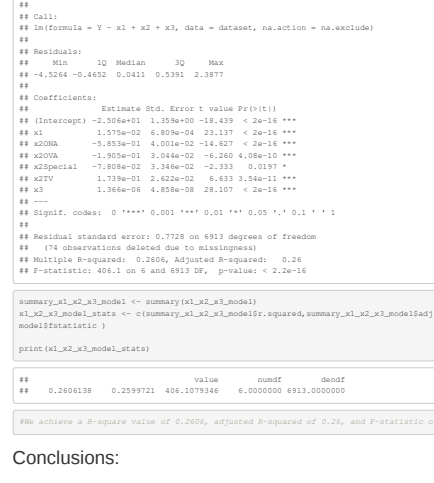
```
#Plotting data on the residual plots:
ggplot(dataset, aes(x = Start_Year, y = full_model_residuals)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "blue")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



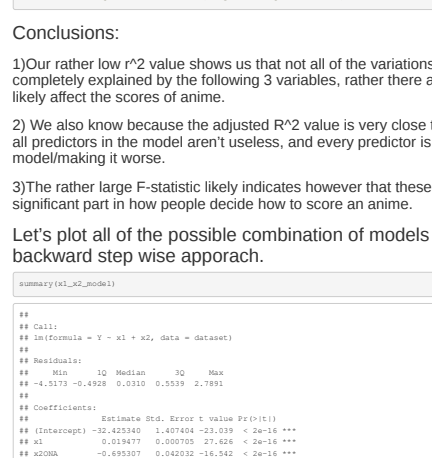
```
ggplot(dataset, aes(x = Type, y = full_model_residuals)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "blue")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



```
ggplot(dataset, aes(x = Members, y = full_model_residuals)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "blue")
```

```
## Warning: Removed 74 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



We fail Start_Year because it's variance was not constant(Funnel-shape), so therefore this would also fail Homoskedasticity.

Outliers:

Looking at all of our residual plots, aside from Type of Anime, the other two predictors(Start_Year, Members) show outliers scatter across its data. This means we need to take into account this when making summaries with our data.

Collinearity:

```
#Looking for values >5:
vif(x1_x2_x3_model)
```

```
##        GVIF Df GVIF^(1/(2*Df))
## x1 1.156774  1        1.075535
## x2 1.219710  4        1.025148
## x3 1.111150  1        1.054130
```

```
#Subsets:
vif(x1_x2_model)
```

```
##        GVIF Df GVIF^(1/(2*Df))
## x1 1.151519  1        1.073088
## x2 1.151519  4        1.017790
```

```
vif(x1_x3_model)
```

```
##       x1       x3
## 1.032267 1.032267
```

```
vif(x2_x3_model)
```

```
##       GVIF Df GVIF^(1/(2*Df))
## x2 1.06585  4        1.008000
## x3 1.06585  1        1.032400
```

Because all of our VIF were less than 5, we can safely assume that all of our variables are not correlated with one another.

Despite a lot of failed conditions, let's run the tests to see what results we can produce:

Full Model:

```
summary(x1_x2_x3_model)
```

```
[illegible]
```

```
summary_x1_x2_x3_model <- summary(x1_x2_x3_model)
x1_x2_x3_model_stats <- c(summary_x1_x2_x3_model$r.squared,summary_x1_x2_x3_model$adj.r.squared,summary_x1_x2_x3_model$fstatistic[1])
print(x1_x2_x3_model_stats)
```

```
##                          value        numdf        dendf
##    0.2404138    0.2399701  406.1075564    6.0000000 6913.0000000
```

```
[illegible]
```

Conclusions:

1)Our rather low r^2 value shows us that not all of the variations in scores is completely explained by the following 3 variables, rather there are other factors that likely affect the scores of anime.

2) We also know because the adjusted R^2 value is very close to the R^2 value, that all predictors in the model aren't useless, and every predictor is not penalizing our model/making it worse.

3)The rather large F-statistic likely indicates however that these variables do play a significant part in how people decide how to score an anime.

Let's plot all of the possible combination of models using the backward step wise apporach.

```
summary(x1_x2_model)
```

```
[illegible]
```

```
summary(x1_x3_model)
```

```
[illegible]
```

```
summary(x2_x3_model)
```

```
[illegible]
```

Conclusions from 2 predictors:

1) We see that all of the models had their adj r^2 < r^2, although by a minimal amount.

2) We can see that the model with B1 and B2 (Start Year and Type) were the weakest of the 3 models presented, showing the lowest R^2, adjusted R^2, and F-stat.

3)If we had to choose one of the 3 models to use, we would use the B1 and B3 model (Start Year and Members) because it has higher summary statistics overall.

Lastly, let's plot out each individual variable.

```
summary(x1model)
```

```
[illegible]
```

```
summary(x2model)
```

```
[illegible]
```

```
summary(x3model)
```

```
[illegible]
```

Conclusions:

The amount of members an anime has is the best when comparing performances of all predictors.(Highest R values and F-stat)

Question 5) Choose the best model of the 7:

When choosing the best models:

1) High R^2/Adj R^2

2) Large F-stat

3) Low AIC/BIC (penalization for model complexity)

The Best Canidates:

```
x1_x2_x3_model<- lm(Y ~ x1 + x2 + x3, data = dataset)
AIC(x1_x2_x3_model)
```

```
## [1] 14079.76
```

```
BIC(x1_x2_x3_model)
```

```
## [1] 14134.47
```

```
summary(x1_x2_x3_model)
```

```
[illegible]
```

```
x1x3_model <- lm(Y~ x1 +x3, data = dataset)
AIC(x1x3_model)
```

```
## [1] 14972.98
```

```
BIC(x1x3_model)
```

```
## [1] 14600.20
```

```
summary(x1x3_model)
```

```
[illegible]
```

```
x3model <- lm(Y~ x3, data = dataset)
AIC(x3model)
```

```
## [1] 17153.84
```

```
BIC(x3model)
```

```
## [1] 17174.4
```

```
summary(x3model)
```

```
[illegible]
```

Based on the summary statistics used, the Full Model is the best model of the bunch.

Question 6) Do people equally prefer the top 5 anime studios? (Chi-squared goodness of fit test)

Setting up the top 5 Studios:

```
studio_counts <- table(dataset$Studios)
top5_studios <- names(sort(studio_counts, decreasing = TRUE)[1:5])
dataset_top5 <- subset(dataset, Studios %in% top5_studios)
```

Hypothesis:

Ho: All studios are equally prefered.

H1: Studios are not equally prefered.

(If p <0.5, we fail H0)

First, we need to meet the assumptions to use the Chi-square:

We assume that the following conditions are true:

1) Data are counts

2) Categories are mutually exclusive

3) Random Sample:

```
[illegible]
set.seed(123)
sampled_top5_studios <- dataset_top5[sample(nrow(dataset_top5), 300), ]
```

Because our data is now randomly sampled, we can continue checking the conditions for Goodness of Fit.

4) Expected Counts:

```
[illegible]
observed <- table(sampled_top5_studios$Studios[c("Toei Animation", "Sunrise", "Madhouse", "J.C.Staff", "Studio Deen")])
print(observed)
```

```
##
## Toei Animation        Sunrise       Madhouse      J.C.Staff    Studio Deen
##             [illegible]
```

```
#Setting up Expected Counts:
expected <- rep(sum(observed) / length(observed), length(observed))
expected
```

```
## [1] 53 53 53 53 53
```

Our expected counts are higher than 5, so we can conduct the test with the following amount of data.

Performing the test(using favorites):

```
[illegible]
observed <- sampled_top5_studios %>%
  group_by(Studios) %>%
  summarise(Total_Favorites = sum(Favorites, na.rm = TRUE))
print(observed)
```

```
## # A tibble: 5 x 2
##   Studios        Total_Favorites
##   <fct>                    <dbl>
## 1 J.C.Staff                [illegible]
## 2 Madhouse                 [illegible]
## 3 Sunrise                  [illegible]
## 4 Toei Animation           [illegible]
## 5 UNKNOWN                  [illegible]
```

```
observed <- observed$Total_Favorites
expected <- rep(1/5, 5)

chisq.test(x = observed, p = expected)
```

```
##
##  Chi-squared test for given probabilities
##
## data:  observed
## X-squared = 198578, df = 4, p-value < 2.2e-16
```

Conclusion:

Because our P-value of close to 0 is less than 0.5, there is sufficent evidence that studios do not equally prefer and enjoy the Top 5 anime Studios.

Question 7) What kind of groups/trends can we observe by clustering Rank, Populuarity, Episodes, and Start Year to Score?(K-Means)

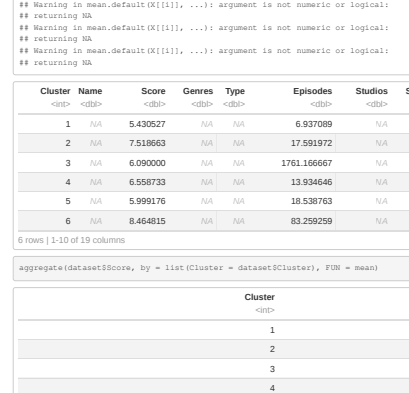
Set up for K-means:

```
#Removing NAs:
k_mean_dataset <- na.omit(dataset[, c("Rank", "Favorites", "Episodes", "Start_Year", "Score")])
#----------
#Scaling data(For distance/centroids):
k_mean_dataset <- scale(k_mean_dataset)
```

Outliers: We are well aware that almost all of our predictors (Aside from rank) are skewed in one way, or atleast have outliers present. This can have the effect of skewing centroids, making this less reliable and potentially inaccurate. Let's take this into account as we test.

Elbow Metrhod for # of Clusters:

```
wss <- sapply(1:10, function(k) {
  kmeans(k_mean_dataset, centers = k, nstart = 10)$tot.withinss
})

plot(1:10, wss, type = "b", pch = 19,
     xlab = "Number of Clusters K", ylab = "Within-Cluster Sum of Squares")
```



While we could make 10 clusters and try to capture as much nuances in our data as possible, it is computationally expensive. Looking at our graph, 6 clusters seem to be the best choice, balancing low WSS with simplicity.

Performing K-Means:

```
set.seed(123)
kmeans_result <- kmeans(k_mean_dataset, centers = 6, nstart = 25)
#---------
#Handling na value with Missing Rows:
valid_rows <- complete.cases(dataset[, c("Rank", "Popularity", "Episodes", "Start_Year")])
# Assign clusters only to specific rows
dataset$Cluster <- NA
dataset$Cluster[valid_rows] <- kmeans_result$cluster
```

Comparing the average scores of all clusters;

```
#Checking Rows with without missing values:
aggregate(dataset[valid_rows, ], by = list(Cluster = dataset$Cluster[valid_rows]), FUN = mean)
```

```
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
```

Cluster	Name	Score	Genres	Type	Episodes	Studios	Source	Rating	Rank
<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	NA	5.430527	NA	NA	6.937089	NA	NA	NA	10593.8698
2	NA	7.518663	NA	NA	17.591972	NA	NA	NA	2008.9572
3	NA	6.090000	NA	NA	1761.166667	NA	NA	NA	8336.8333
4	NA	6.558733	NA	NA	13.934646	NA	NA	NA	6076.8885
5	NA	5.999176	NA	NA	18.538763	NA	NA	NA	8434.0775
6	NA	8.464815	NA	NA	83.259259	NA	NA	NA	343.8519

6 rows | 1-10 of 19 columns

```
aggregate(dataset$Score, by = list(Cluster = dataset$Cluster), FUN = mean)
```

Cluster	x
<int>	<dbl>
1	5.430527
2	7.518663
3	6.090000
4	6.558733
5	5.999176
6	8.464815

6 rows

Observations based on Cluster Summaries:

Cluster #1: A low rated, low episode anime with a terrible rank, and almost no fans.

Cluster #2: Above Average scored, average episode Anime with a strong rank and large amount of fans.

Cluster #3: A below average rated, extremely long episode anime, with a decent rank and a niche fanbase.

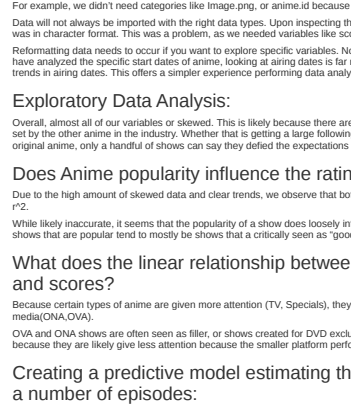
Cluster #4: Average Scored, above average episode Anime with a average rank and fairly little amount of fans (Members).

Cluster #5: A fairly low rated, above average length, unpopular, anime with little to no favorites.

Cluster #6: A high rated, long anime with a prestine rank, popular show with a large fanbase.

Visualizing this:

```
fviz_cluster(kmeans_result, data = k_mean_dataset,
             geom = "point", ellipse.type = "norm", main = "K-Means Clustering of Anime")
```



Conclusions:

1) We can see two distnct clusters that are larger than the rest, 3 and 6. This means that these two clusters contain more data points, and have a high cluster variance, capturing a wide range of values.

2) Overlapping data in all other clusters indicate that our clusters may be similar, and our variables do not define and strong differentation between the clusters and groups.

3) Because our data was highly skewed and overlapping this is a good indicator that K-means may not be the best algorithm to capture our data.

Part 4) Conclusions:

Data Cleaning:

When working with a large dataset like MyAnimeList, we will run into scenarios where we obtain more variables than we need. For example, we didn't need categories like Image.png, or anime.id because for our data analysis, they were essentially useless.

Data will not always be imported with the right data types. Upon inspecting the dataset for the first time, I noticed that everything was in character format. This was a problem, as we needed variables like score or number of favorites in integers or floats.

Reformatting data needs to occur if you want to explore specific variables. Normalizing your data is important, as while I could have analyzed the specific start dates of anime, looking at airing dates is far more broad, and offers a better retrospective of the trends in airing dates. This offers a simpler experience performing data analysis.

Exploratory Data Analysis:

Overall, almost all of our variables or skewed. This is likely because there are only a few shows that break the mold of the trend set by the other anime in the industry. Whether that is getting a large following, long running series, or making some of the first original anime, only a handful of shows can say they defied the expectations of what anime is.

Does Anime popularity influence the rating in anime?:

Due to the high amount of skewed data and clear trends, we observe that both variables fail all of the conditions need to perform r^2.

While likely inaccurate, it seems that the popularity of a show does loosely influence the rating of a show.This is likely because shows that are popular tend to mostly be shows that a critically seen as "good" rather than bad.

What does the linear relationship between types of anime media, and scores?

Because certain types of anime are given more attention (TV, Specials), they usually perform better than other media(ONA,OVA).

OVA and ONA shows are often seen as filler, or shows created for DVD exclusives, or ones for streaming platforms, and because they are likely give less attention because the smaller platform performs worse than TV or specials.

Creating a predictive model estimating the score of an anime given a number of episodes:

While performing a t-test at a 95% confidence level for episodes and the score of anime, we see that there is statistical sufficient evidence to prove the number of episodes does impact the rating a show achieves(P-value of 0.000001).

This is likely because when a story is given more time, it is able to explain more of its story, while also cultivating more of a connection with its fan base. Great examples are shows like Naruto and One Piece.

When creating a predictive model for a 12 episode anime, the rating of the anime is 6.553, roughly the average of what an anime is scored.

Fit a model with Score, Start Year, Type, and Members.

While most of our conditions failed, if we were to guess a model with the following variables, we would choose the full model of all of the possible models available due to thee AIC, BIC, f-statistic, r^2/scores. This means if we wanted to choose a model to predict the following variables, the full model will give us the most accurate results.

Do people equally prefer the top 5 anime studios?:

Using an alpha of 0.05, because our p-value is close to 0, we have sufficient evidence that people do not prefer anime studios equally.

This is likely because each studio has is own style, and varying qualities. Some release more consistent quality shows than others.

What kind of trends and groups can we find by clustering rank, popularity, episodes, and start year to the response variable, score?

Observing our 6 groups of data, it seems that not only does our data overlap between different clusters but the variations of groups 3 and 6 vary far more than the other 4. The K-means algorithm may not be the best when separating these groups using the following variables.

Future Improvements:

1) Use an updated dataset. Including the shows of 2024 could potentially yield slightly different results, and keep our analysis further up-to-date.

2) Select Variables that do not fail conditions. Because 2 of our conclusions were inaccurate due to poor variable selection, I should ensure the variables pass all conditions to ensure statistically useful conclusions.

3) Implement more/different statistical techniques. If I were to revisit this project, I want to include more machine learning techniques like LOOCV, or other simpler statistical tests like Confidence intervals and tests.

4) Perform lookups. I want to find entries with certain qualities about them, whether it is finding my favorite shows in this database, or shows with a certain genre, I can further my exploratory data analysis by doing the following.

# Part 5) Closing Comments:

While creating this project, I never would have expected to make so many interesting discoveries about anime:

I would have never expected so many aspects of anime to be skewed, so many variables to influence the rating of shows, and how much data goes into each anime entry.

The largest lesson learned from this process was how much time data scientist learn just through cleaning data itself. Whether that was deciding which variables to delete, normalizing columns, or changing data types, this was by far the most time consuming portion. It reminds me that while getting the results may be easy(writing functions to obtain those results), the preparation for the testing procedure is the first and biggest step.

This has been a long, yet fun project that combined my love for anime with my passion for data science. I had a blast exploring the different ways I could analyze data through the various statistical techniques learned throughout courses. I hope to revisit this project in the future, and continue to draw more conclusions with this expansive dataset.